Palatial Pools

OpenManus User Guide

1. Introduction and Access

OpenManus is an autonomous agent framework deployed within the Palatial Pools infrastructure. It is designed to interpret high-level instructions, plan multi-step tasks, and execute them independently. By leveraging capabilities such as web browsing, file operations, and code generation, OpenManus can automate complex workflows and assist in development tasks.

This guide provides the necessary instructions to access, utilize, and optimize your interactions with OpenManus.

How to Access and Log In

Access to OpenManus is restricted via Basic Authentication through the Nginx reverse proxy to ensure the security of the Palatial Pools environment.

Navigate to the designated URL for the service: `https://openmanus.palatialpools.com.au`. Upon accessing the page, your browser will prompt you for credentials.

Enter the authorized username and password provided for the OpenManus interface. Once authenticated, you will gain access to the framework and can begin initiating autonomous tasks.

2. Step-by-Step Usage Instructions

Operating OpenManus involves providing clear, objective-oriented prompts and monitoring its progress as it plans and executes the necessary steps.

1. **Define the Objective**: Clearly define the goal you want OpenManus to achieve. This could range from researching a topic on the web to generating a specific script.
2. **Provide the Prompt**: Enter your instructions into the OpenManus interface. The prompt should be as explicit and structured as possible to guide the agent effectively.
3. **Monitor Execution**: OpenManus will begin planning its approach and executing steps. You can observe its thought process, the actions it takes, and any intermediate results it produces.
4. **Intervene if Necessary**: If the agent appears to be stuck in a loop or deviating from the intended objective, you may need to interrupt the process and provide a refined prompt.
5. **Review the Outcome**: Once OpenManus completes the task, review the final output or the changes it has made to ensure they align with your requirements.

3. Task Capabilities

OpenManus is equipped to handle a variety of tasks that require planning, execution, and interaction with the environment.

What Kinds of Tasks It Can Handle

The framework is particularly suited for tasks that involve research, file manipulation, and code generation.

Task Category	Examples
Autonomous Planning	Breaking down a high-level objective into a series of actionable steps.
Web Browsing	Navigating websites, extracting information, and compiling research data.
File Operations	Creating, reading, updating, and deleting files within its accessible environment.
Code Generation	Writing scripts or code snippets based on provided requirements and context.

4. Best Practices for Prompting

Due to its autonomous nature and occasional stability issues, effective prompt engineering is critical when using OpenManus.

The most important practice is to provide highly structured and explicit instructions. Break complex objectives down into smaller, logical steps. Instead of asking OpenManus to complete a large project in a single prompt, guide it through the process sequentially.

Always specify constraints and the desired format of the output. If a task must not modify certain files or must adhere to a specific coding standard, state these requirements clearly in the prompt. This level of detail helps prevent the agent from making incorrect assumptions or taking unintended actions.

5. Common Issues and Resolutions

While OpenManus is a capable framework, you may encounter operational issues, particularly given its relative immaturity compared to other solutions.

Execution Loops

A common issue with OpenManus is getting stuck in an execution loop, where the agent repeatedly attempts the same failing action without progressing toward the objective. If you observe this behavior, the most effective resolution is to interrupt the current task. Restart the process with a refined, more structured prompt that explicitly guides the agent past the point of failure.

Lack of Sandboxing

Unlike some other agent frameworks, OpenManus does not currently implement built-in sandboxing. This means it executes its tasks directly within its containerized environment or on the host system, depending on its configuration. You must be mindful of this constraint when assigning tasks that involve executing untrusted code or modifying system configurations. Ensure the agent is only assigned tasks that are safe to execute within its operational context.

© 2026 Palatial Pool Services Pty Ltd. Internal Technical Documentation.